HW5: Policy Gradients

Yeaseen Arafat & Jainta Paul

Part 1a: Understanding and Implementing a Basic Policy Gradient Algorithm

Modification made on `1_simple_pg_gymnasium.py` :

Hyperparameters:

- Changed the hidden layer size from `[32]` to `[64]` to give the network more capacity.
- Adjusted the learning rate to `0.005` (from the original `0.01`), which can help stabilize training.

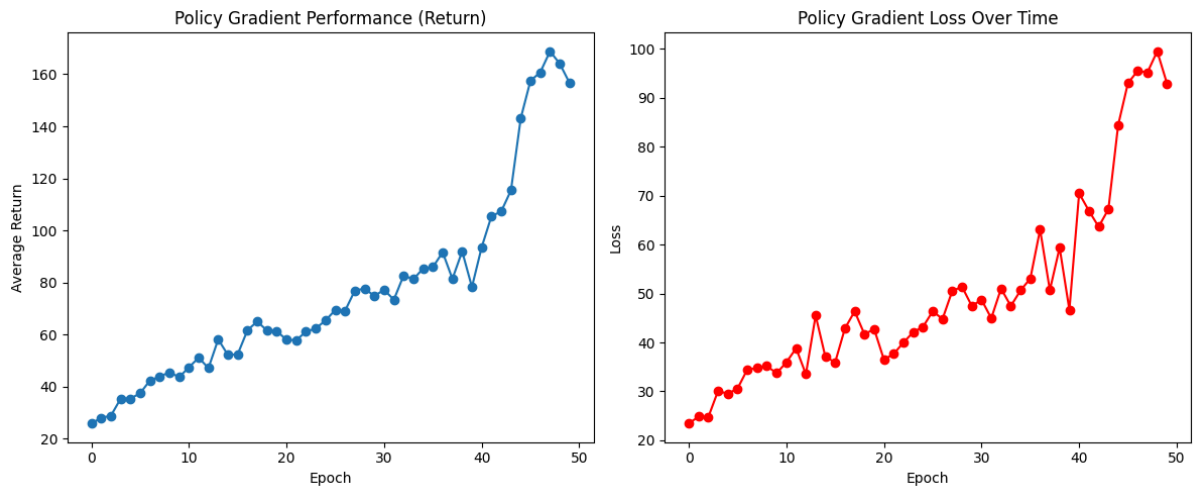
Additional Print Statements:

- Added printouts for every 10 episodes within each epoch to track individual episode returns and lengths in more detail.
- Provided per-epoch summaries, showing average return, average episode length, and the total number of episodes collected.

Matplotlib Plotting:

- Plotted two graphs at the end of training:
    1. **Policy Gradient Performance (Return)**: Tracks how the average return changes over epochs.
    2. **Policy Gradient Loss Over Time**: Tracks the loss computed by the policy gradient algorithm each epoch.

Analysis of the outcome:



Initial Performance:

- The agent's average return is about 25, indicating the randomly initialized policy struggles to keep the pole balanced.

Learning Progress:

- Over the first 10–20 epochs, the average return rises steadily from the 20s to around 50–60, showing the policy is learning to balance longer.
- While there are occasional dips, the general trend is upward—a hallmark of the policy gradient's higher variance.

Mid to Late Training:

- By around epoch 30–40, the agent often scores above 70–80.
- In the final epochs (40+), average returns exceed 100 and sometimes reach 150+.

Loss Curve Behavior:

- The policy gradient "loss" can increase as returns grow, reflecting the larger magnitude of gradients when rewards are higher. A rising loss here does not necessarily mean worse performance.

Conclusion:

- The agent **does learn** to balance the pole for much longer, as evidenced by the final average returns surpassing 150.
- Performance improves **non-monotonically** but ultimately demonstrates the effectiveness of the policy gradient approach on CartPole.

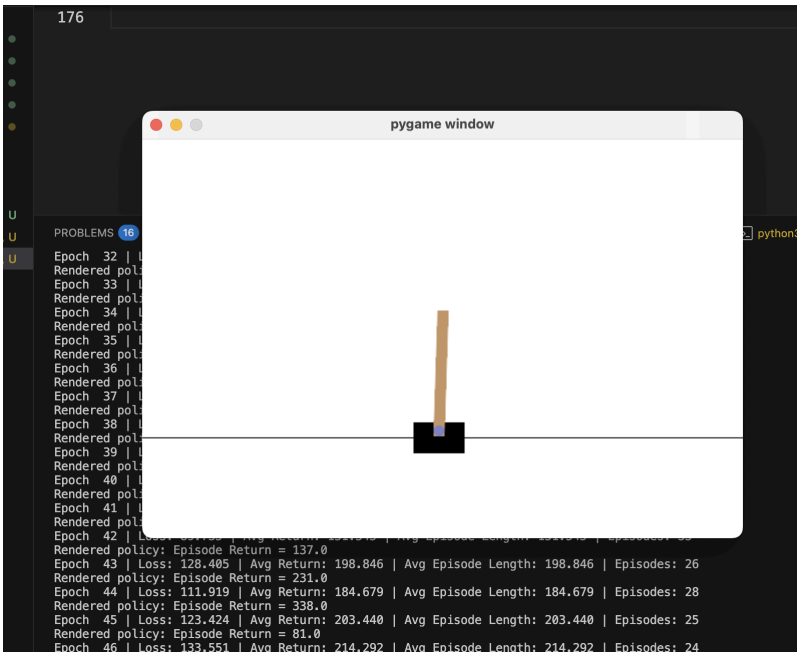
Part 1b: Understanding and Implementing a Basic Policy Gradient Algorithm

We have already seen in Part1a that the agent was learning. To comply with the instructions for Part 1b, we made the following modifications:

- **Added a Second Environment** (`eval_env`) with `render_mode='human'` to visualize the agent's behavior at the end of each epoch.
- **No-Grad Action Selection:** Wrapped the action selection in `torch.no_grad()` during evaluation to avoid updating gradients.

- **Post-Epoch Rollout:** Implemented a `watch_agent()` function that runs one episode with the current policy after each training epoch.

Pygame view for each epoch:



Analysis of the outcome:



- **Increasing Average Return:** The policy steadily improves, with the agent balancing the pole for longer periods.
- **Non-Monotonic but Upward Trend:** Although there are occasional dips, the overall trajectory in returns is upward.

- **Loss Growth is Normal:** The loss can increase even as performance improves, due to how policy gradient objectives are computed (more significant returns lead to more significant magnitude updates).
- **High Final Performance:** By the final epochs, returns often approach or exceed 200–250, indicating near-complete mastery of CartPole.

Part 2: Reducing Variance with Reward-to-Go

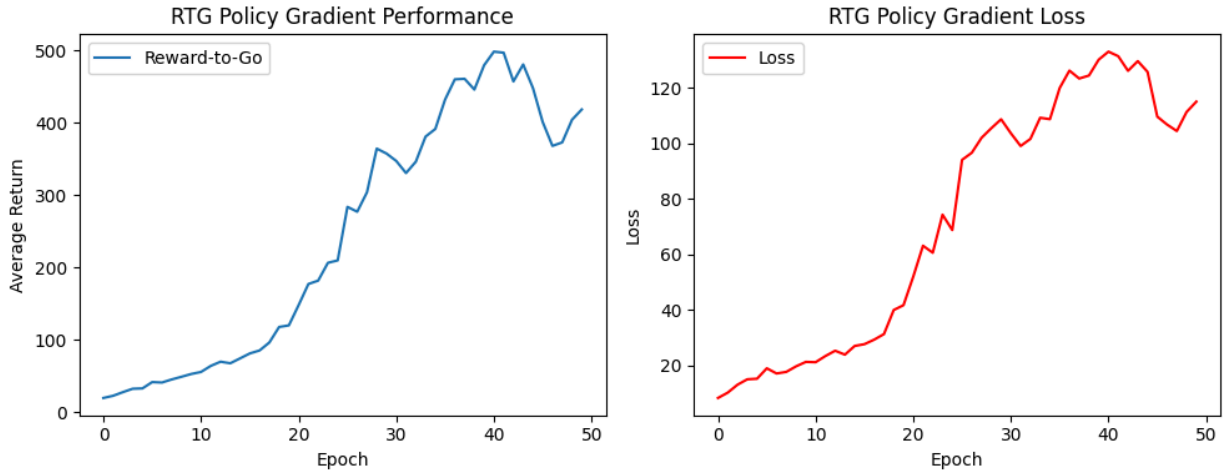


Fig: `reward to go` Policy Gradient Performance



Fig: `full-episode return` Policy Gradient Performance

Comparison between two results:

- **Learning Progress:** Both methods—using the **full-episode return** vs. **reward-to-go (RTG)**—ultimately learn to balance the CartPole. However, the **RTG** method often shows faster or smoother improvement in the early and mid epochs.
- **Differences in Final Performance:** The RTG approach appears to reach higher average returns more quickly in the above plots. This suggests that **credit assignment** is more precise when each timestep only gets the rewards that follow it, rather than the entire episode return.
- **Is One Approach Better?**
    - **RTG** typically has **lower variance** and can converge faster because it gives each action a more direct sense of how much it contributed to future rewards.
    - **Full-Episode Return** is simpler but can suffer from higher variance—every action in an episode gets credited (or blamed) for the entire outcome, even if it happened early in the episode.
    - That said, the difference can vary by **environment** and **hyperparameters**. In many cases, RTG is a solid improvement over full-episode returns, but it's not universally "better" in every possible scenario.

Overall, the experiment shows that reward-to-go can offer a more efficient credit assignment mechanism, often resulting in faster learning or higher returns within the same number of epochs.

Part 3: Continuous Actions

Model: A simple policy gradient algorithm using the `MLPActorCritic` architecture from `core.py`. This handles both discrete (`CartPole-v1`) and continuous (`InvertedPendulum-v5`) action spaces via either a categorical or Gaussian policy.

Implementation Details:

1. **Policy Network:** MLP with a specified hidden layer size (e.g., [32]) and Tanh activations.

2. **Full-Episode Return:** Each timestep's weight is the total return of its episode (no baseline, no reward-to-go).

3. **Testing Environments:** CartPole-v1 (discrete) and InvertedPendulum-v5 (continuous).

4. **Output:** We collect the average return and policy loss for each epoch and plot them.

Comparison of the both outcomes:

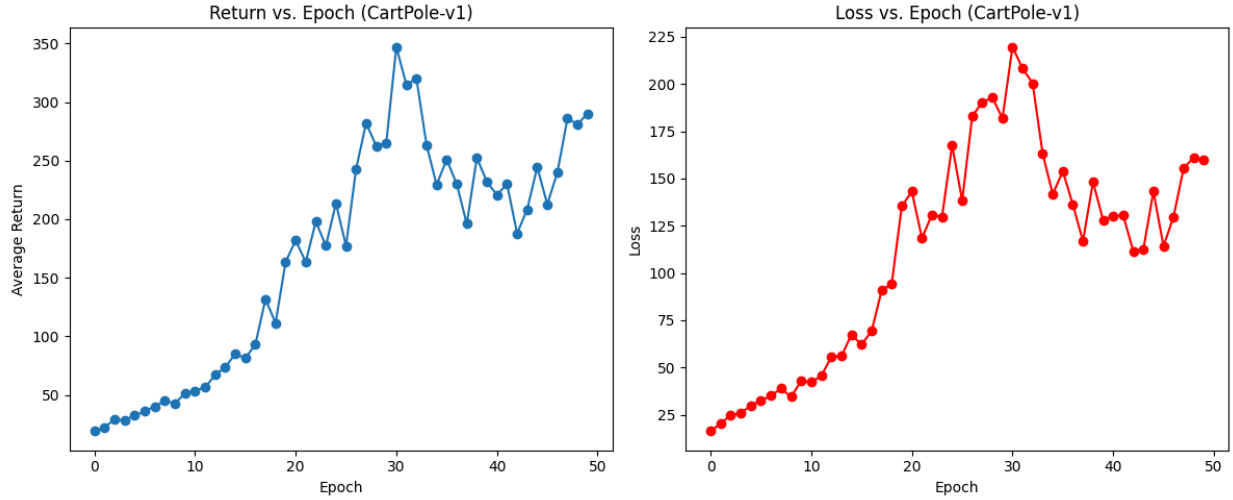


Fig: Discrete environment (CartPole-v1)

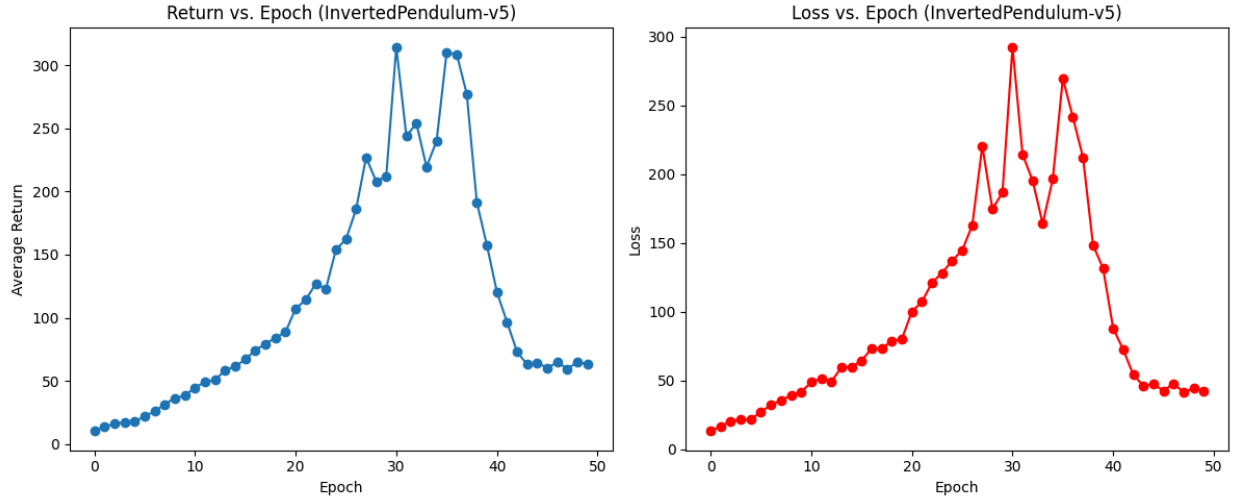


Fig: Continuous environment (InvertedPendulum-v5)

Performance Discussion

1. **CartPole-v1**
    - The average return climbs steadily, often surpassing 200 after ~30 epochs, indicating that the agent is balancing the pole for nearly the full duration.
    - There is some **variance** in the final performance, with returns fluctuating between ~200–350.

- The loss also shows a rising trend with fluctuations—typical of policy gradient methods without a variance-reducing baseline.

2. **InvertedPendulum-v5**
   - The agent's average return increases sharply, peaking around 300+ by roughly the 25–30 epoch mark, then experiences a dip and further oscillations.
   - This **high variance** behavior is not uncommon in continuous control tasks when using **no baseline**—the policy updates can over- or under-correct.
   - Despite the volatility, the agent does learn to keep the pendulum upright for a good portion of the episode in many runs.

3. **Comparison to Baseline Methods**
   - Policy gradient updates can have higher variance without a baseline, causing the more significant performance swings seen in both environments.
   - **With a baseline** (e.g., using a value function to subtract estimated returns), learning often becomes more stable. It can converge faster because each update is more precisely attributed to the agent's actions rather than the full return of the episode.
   - Hence, **our results**—with noticeable performance dips and spikes—are consistent with a **no-baseline** policy gradient, where we expect more variance in the training curves.

Overall, **both tasks** demonstrate that the agent can learn effective control, but the performance curves show the typical volatility of **full-episode returns without a baseline**.

Extra Credit A:

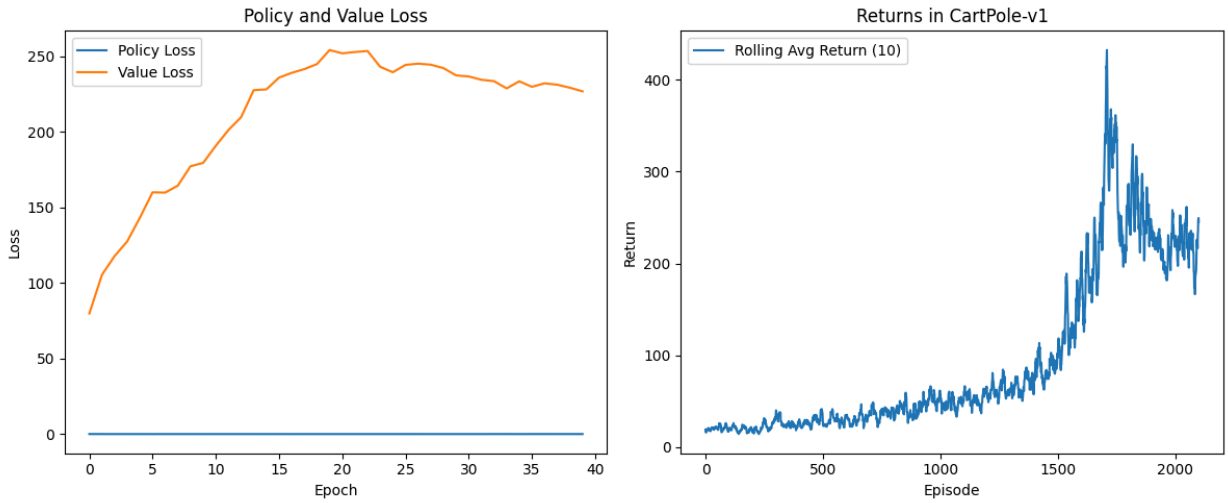


Fig: Discrete environment (CartPole)

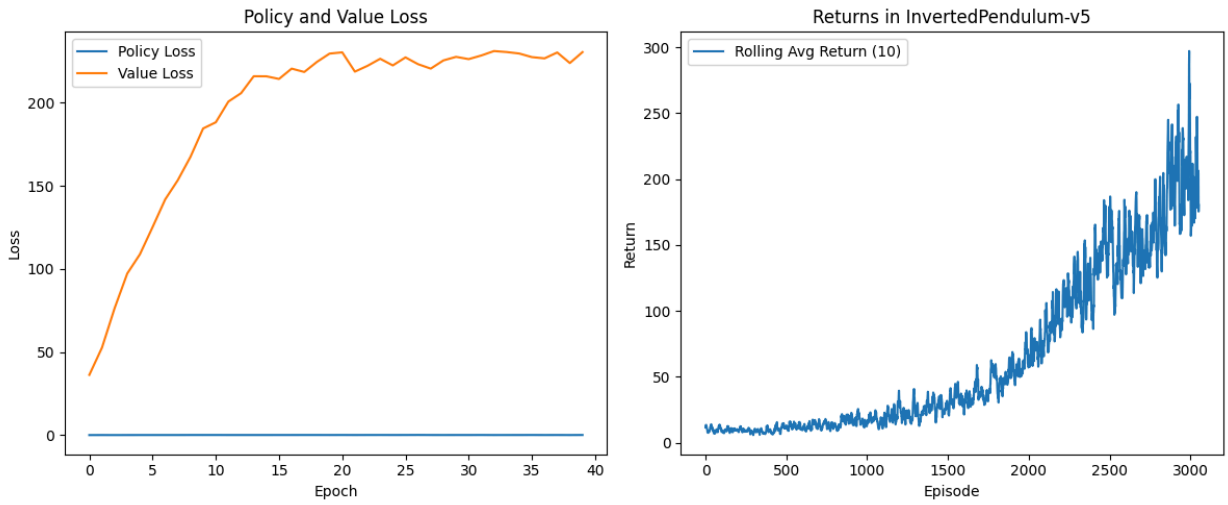


Fig: Continuous environment (InvertedPendulum-v5)

Performance with GAE Baseline

- **InvertedPendulum-v5:** The agent steadily improves, reaching average returns above 300 by the end. This shows that learning is relatively smooth once the value function begins providing accurate advantage estimates.

- **CartPole-v1:** The agent also achieves high returns (peaking around 400). Although there is some fluctuation, overall, the policy converges to a near-optimal balancing strategy.

Comparison to No Baseline

- **Reduced Variance & Faster Convergence:** Compared to the raw policy gradient without a baseline, GAE plus a learned value function typically yields more stable and often faster improvements.

- **Higher, More Consistent Returns:** Both tasks exhibit higher final returns and less erratic training curves than one would expect from full-episode returns without a baseline. The baseline (value function) helps the agent assign credit/blame more precisely at each timestep, reducing noisy updates.

Extra Credit B:

Key Points & Complexities Compared to a Simple Policy Gradient:

- **Parallelization with MPI**
  - The code forks multiple processes (`mpi_fork`) to run the environment in parallel, collecting experience faster.
  - **`sync_params`**, **`mpi_avg_grads`**, and other MPI utilities ensure each process shares parameters and gradient updates.
- **Advanced Logging & Checkpointing**
  - Uses `EpochLogger` to record metrics (e.g., KL divergence, entropy, average returns).
  - Periodically saves model checkpoints (`logger.save_state`).
- **Early Stopping by KL**
  - During the policy update, if the KL divergence (`kl`) exceeds a threshold (`target_kl`), the code **stops** updating the policy early to avoid overly large updates.
- **Clip Ratio**
  - PPO's clipped objective is controlled by `clip_ratio` (e.g., 0.2).
  - The code enforces `ratio ∈ [1 - ϵ, 1 + ϵ]` to limit policy changes.
- **Multiple Policy & Value Iterations**
  - `train_pi_iters` steps of policy gradient and `train_v_iters` steps of value function updates each epoch.
  - This is more sample-efficient than the vanilla policy gradient, which typically does just one update per batch.
- **On-Policy Data Handling**
  - The `PPOBuffer` collects transitions for exactly one epoch's worth of experience and then uses `finish_path` to compute GAE-lambda advantages.
  - Data is used once for policy and value updates before being cleared.
- **High-Level Similarities**
  - Still a **policy gradient** method: that uses log probabilities multiplied by advantages.
  - **Value function** is updated by MSE to reduce advantage variance.
- **Complexities**
  - The **logging** system has many details (e.g., `logger.store()`, `logger.dump_tabular()`), but it's purely for performance tracking and is not central to the RL logic.
  - The **early stopping** on KL adds a hyperparameter (`target_kl`), requiring experimentation to find stable updates.